Primer: Deep Maximum Causal Entropy IRL

econirl package documentation

1 Problem

Linear MCE-IRL (Ziebart, 2010) uses a reward $R(s,a) = \theta^\top \phi(s,a)$ and solves the maximum causal entropy control problem under known transitions. Wulfmeier, Ondruska, and Posner (Wulfmeier et al., 2015) replace the linear state reward with a neural map from supplied state features to rewards. The Bellman recursion and occupancy matching logic stay the same; only the reward parameterization changes.

The package implementation validated here is `MCEIRLNeural`. The primary Shapeshifter validation uses a frozen nonlinear neural reward, known stochastic transitions, supplied state encodings, and action 0 as a zero-reward anchor. The gated artifact for neural-reward truth is the learned raw `reward_matrix`, not a projected finite θ. The raw neural network weights are not treated as finite economic parameters. When the truth is linear in a supplied feature matrix, the post-fit projection is reported and parameter gates apply only when that projection is numerically identifiable. Counterfactual gates apply the same affine IRL reward normalization convention before adding intervention deltas.

2 Objective

Table 1: Common DDC notation shared across all primers.

Symbol	Name	Definition
$s \in \mathcal{S}$	State	Discrete or continuous
$a \in \mathcal{A}$	Action	Finite: $\lvert\mathcal{A}\rvert = J$
s'	Next state	Drawn from $P(s' \mid s,a)$
$\beta \in [0,1)$	Discount factor	
$\sigma > 0$	Scale parameter	Logit noise scale ($=1$ by convention)
$\theta \in \Theta \subset \mathbb{R}^{d_\theta}$	Structural parameters	Unknown
$\phi(s,a)$	Feature vector	Known functions of (s,a), dimension K
$u(s,a;\theta)$	Flow utility	$\theta^\top \phi(s,a)$ (linear-in-parameters)
$P(s' \mid s,a)$	Transition kernel	Known or estimated from data
$Q(s,a)$	Choice-specific value	$u(s,a;\theta) + \beta \sum_{s'} P(s'\lvert s,a)\, V(s')$
$V(s)$	Integrated value	$\sigma \log \sum_a \exp(Q(s,a)/\sigma)$
$\pi(a \mid s)$	Choice probability	$\exp\big((Q(s,a) - V(s))/\sigma\big)$
τ	Trajectory	$(s_0, a_0, s_1, a_1, \ldots)$ from one agent
$\mathcal{D}$	Panel data	Collection of N observed trajectories

For a neural reward R_ψ, the soft control problem is

$$\max_\pi \; \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \beta^t \big(R_\psi(s_t,a_t) + \sigma \mathcal{H}(\pi(\cdot \mid s_t)) \big) \right]. \tag{1}$$

Given a current reward surface, the backward pass solves

$$Q(s,a) = R_\psi(s,a) + \beta \sum_{s'} P(s' \mid s,a) V(s'), \tag{2}$$

$$V(s) = \sigma \log \sum_a \exp(Q(s,a)/\sigma), \tag{3}$$

$$\pi(a \mid s) = \exp((Q(s,a) - V(s))/\sigma). \tag{4}$$

The forward pass computes the normalized discounted state-action occupancy $\mu_\pi(s,a)$. The neural update uses the same occupancy mismatch as linear MCE-IRL:

$$\nabla_\psi L = \sum_{s,a} \big(\mu_\pi(s,a) - \mu_D(s,a)\big) \nabla_\psi R_\psi(s,a). \tag{5}$$

3 Identification

Neural rewards inherit the usual IRL normalization problem. Additive shifts, positive rescaling, and richer potential-like reward transformations can leave the induced policy nearly unchanged (Ng et al., 1999; Kim et al., 2021). In action-dependent DDC cells we therefore anchor action 0 at zero reward. The validation separates four objects:

- raw neural reward matrix: the gated object when the truth is a frozen neural reward;
- neural weights: diagnostic only, not structural parameters;
- projected reward: used for finite linear truths when the projection basis is well conditioned;
- induced behavior: policy, value, Q, occupancy, and counterfactual performance under the recovered reward map.

The success claim below is nonlinear reward-map recovery over supplied state encodings and known transitions. It is not a claim that raw network weights are uniquely identified. It is also not a claim that the package learns raw Objectworld-style spatial features with a convolutional FCNN; that would require a convolutional reward network rather than the current MLP over supplied encodings.

4 Algorithm

Algorithm 1: Deep MCE-IRL with Anchored Reward-Map Validation

Input: Demonstrations, known transitions P, discount β, state encoder, optional action or absorbing-state anchor

Compute discounted empirical occupancy $\mu_D(s,a)$ from trajectories
Initialize neural reward network R_ψ
for *epochs* **do**
  Compute anchored reward matrix $R_\psi(s,a)$
  Solve the soft Bellman equations for V, Q, and π
  Compute model occupancy $\mu_\pi(s,a)$ from the empirical initial distribution
  Update ψ with surrogate loss $\sum_{s,a} R_\psi(s,a)(\mu_\pi(s,a) - \mu_D(s,a))$
end
If the truth is a finite linear feature model and the projection is identified, compute $\hat{\theta} = \arg\min_\theta \|R_\psi - \Phi\theta\|^2$

Output: Reward matrix, optional projected parameters, policy, value, Q, and counterfactual metrics

5 Results

5.1 Generated Known-Truth Results

This section is generated by `deep_mce_irl_run.py` from the `experiments.known_truth` harness. The primary validation target is an anchored raw nonlinear reward map learned by `MCEIRLNeural` from demonstrations under known stochastic transitions and supplied state encodings. The Shapeshifter neural cells use action 0 as the zero-reward anchor. Neural network weights are not treated as structural parameters. When the Shapeshifter truth has a frozen neural reward, the gated artifact is the learned `reward_matrix`; finite projected parameters are gated only when the projection basis is numerically identifiable.

Table 2: Deep MCE-IRL known-truth validation cells.

Cell	Role	Truth	States	Actions	Individuals	Periods
Canonical projected reward	sanity	projected_linear_reward_matrix	21	3	1,000	80
Neural reward	primary	raw_neural_reward_matrix	32	3	2,000	80
Neural features	finite-theta check	projected_linear_reward_matrix	32	3	2,000	80
Neural reward + features	stress test	raw_neural_reward_matrix	32	3	2,000	80

Table 3: Deep MCE-IRL structural recovery metrics.

Cell	Gates	Occ. resid.	Reward nRMSE	Policy TV	Value nRMSE	Q nRMSE	Param cosine	Type A/B/C regret
Canonical projected reward	11/11	0.0028	0.0648	0.0111	0.0923	0.0629	0.9891	0.0105/0.0775/0.0011
Neural reward	9/9	0.0018	0.0436	0.0047	0.0778	0.0442	—	0.0016/0.0015/0.0019
Neural features	9/9	8.4709e-04	0.0539	0.0024	0.0054	0.0550	0.0369	2.4006e-04/2.5929e-04/1.5374e-04
Neural reward + features	9/9	0.0016	0.0380	0.0071	0.0141	0.0429	—	0.0024/0.0023/2.2735e-04

Cell `canonical_low_state_only` passes 11/11 hard gates. Cell `deep_mce_neural_reward` passes 9/9 hard gates. Cell `deep_mce_neural_features` passes 9/9 hard gates. Projected finite-theta statistics are diagnostic in this cell because the projection condition number is 478.680. Cell `deep_mce_neural_reward_features` passes 9/9 hard gates.

6 Code

The estimator is implemented in `src/econirl/estimators/mceirl_neural.py`. The shared known-truth adapter and gates live in `experiments/known_truth.py`. The generated tables are produced by:

```
PYTHONPATH=src:. python \
  papers/econirl_package/primers/deep_mce_irl/deep_mce_irl_run.py \
  --enforce-gates
```

References

Kuno Kim, Shivam Garg, Kirthevasan Shiragur, and Stefano Ermon. Reward identification in inverse reinforcement learning. *ICML*, 2021.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations. In *ICML*, pages 278–287, 1999.

Markus Wulfmeier, Peter Ondruska, and Ingmar Posner. Maximum entropy deep inverse reinforcement learning. In *NIPS Deep Reinforcement Learning Workshop*, 2015.

Brian D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.